Kravspecifikation

Projektnavn: RentIt
System: Web Applikation - Udlejeningsplatform
Gruppe: Chanakya Joshi, Subarna Gurung, Marko Dankovic
Uddannelse: Tec Eux Data og Kommunikation
Afleveringsdato: 27.03.2026

RenIt

Inhold

Indledning 1
Problemformulering 1
Problemformuleringsspørgsmål 1
Baggrund 2
Forretningsmæssige gevinster 2
Set fra brugerens side 2
Afgrænsning 3
Fremtid (mulige udvidelser) 3
Aktørkontekst diagram 4
Aktør beskrivelse 5
Usecase Diagram 7
Krav om funktionalitet 8
Case beskrivelser 10
Systembeskrivelse 11
Programmets begrænsninger 11
Programmets fremtid 11
Brugerprofil 11
Krav til udviklingsforløbet 12
Frontend Krav 12
Backend krav 12
Specifikke teknologi krav 13
Ikke Funktionelle krav 13
Testing 13
Integration testing 13
User testing 14
Sikkerhed (login, hashing, kryptering, SSL) 14
Password hashing og login-sikkerhed 14
JWT + rollebaseret adgang 14
Kryptering af data (database) 14
SSL/HTTPS certifikat 15
Ekstra sikkerhed 15
Kvalitetsfaktor 15
Use case 15
Wireframes 20
GANTT Diagram 25

Indledning

Formålet med projektet er at lave en webapp, hvor brugere kan leje og udleje ting som fx bøger, film, spil, verktøjer og andre genstande. Appen skal gøre det nemt at oprette opslag, finde ting man vil leje, sende leje forespørgsler og holde styr på afleveringsdatoer. Samtidig skal der være et admin-system, så nye opslag først bliver godkendt, før de vises for alle.

Problemformulering

Vores problem er, at der ikke findes en nem og “tryg” måde for private at leje ting af hinanden (som bøger, film, spil, værktøjer og mm.) uden at det bliver rodet eller usikkert. Mange har ting liggende derhjemme, som andre kunne bruge, men man tør ikke altid låne det ud, fordi man ikke har overblik, regler eller en måde at sikre sig (fx med depositum). Samtidig kan det også være svært for lejeren at finde ting hurtigt og holde styr på afleveringsdato.

Derfor vil vi lave en web app, hvor brugere kan:

- oprette en konto
- lægge deres egne ting op til udlejning
- browse og søge i ting som andre har lagt op
- sende en leje-forespørgsel med et lille depositum
- og få et overblik over hvad de lejer og hvornår de skal aflevere

For at undgå scam og dårligt indhold, vil vi også have en admin rolle, som kan godkende eller afvise nye opslag, før de bliver vist offentligt. Admin skal også kunne se alle brugere og alle opslag/lejer, så man kan tjekke hvis der er problemer.

Problemformuleringsspørgsmål

1. Hvordan kan vi lave et login og rollesystem (User/Admin), så kun rigtige brugere kan oprette opslag og sende leje-forespørgsler, og så admin har de rigtige rettigheder?

2. Hvordan kan vi lave admin-godkendelsen af nye opslag, så kun godkendte items bliver vist i browse sektionen, og brugeren altid ser de rigtige opslag?

3. Hvordan kan vi lave en søge og browse-funktion, så brugere nemt kan finde det de leder efter (fx kategori, navn, tilgængelighed), og samtidig holde systemet hurtigt og overskueligt?

4. Hvordan kan vi håndtere leje processen med forespørgsel og depositum, så det føles trygt for udlejeren, og så lejeren altid kan se status og afleveringsdato?

Baggrund

Forretningsmæssige gevinster

Der findes mange genstande, især værktøj og maskiner, som folk kun bruger få gange om året – fx en plæneklipper, boremaskine, højtryksrenser eller stige. For mange giver det ikke mening at købe dyrt udstyr, når behovet er sjældent. Vores løsning gør det muligt at leje i stedet for at eje, og dermed udnytte ressourcer bedre. Platformen skaber værdi for begge sider: udlejeren kan tjene penge på ting der ellers bare står og samler støv, og lejeren kan spare penge ved kun at betale for den periode, hvor tingene faktisk bruges.

Samtidig giver appen en mere struktureret og tryg proces end “skriv i en Facebook-gruppe”. Med en fast pris pr. dag, et depositum og et system der kan vise lokation og status på lejeaftaler, bliver det nemmere at lave aftaler og undgå misforståelser. For platformen betyder det også, at den kan skaleres: jo flere brugere og opslag, jo mere værdi skaber den, og på sigt kan man udvide med fx betalingsløsning, premium-funktioner eller samarbejde med lokale foreninger/kommuner.

Set fra brugerens side

Set fra brugerens perspektiv handler systemet om at gøre det hurtigt at finde og leje noget tæt på. En bruger opretter en konto og kan med det samme browse opslag. Når lokation er aktiveret, kan brugeren filtrere efter “nær mig”, så de kun ser ting i et område de kan hente uden besvær. Brugeren kan fx søge “plæneklipper”, se pris pr. dag og depositum og vælge en periode. Hvis brugeren vil leje, kan de sende en forespørgsel direkte fra opslaget og derefter chatte med udlejeren for at aftale detaljer som afhentningstidspunkt og adresse. Det gør processen mere personlig og praktisk, fordi mange lejeaftaler kræver koordinering. Brugeren kan også selv være udlejer: oprette et opslag, sætte pris og depositum og vælge lokation, så andre kan finde det. Når en aftale er lavet, kan begge parter følge status i systemet, og lejeren kan se forfaldsdato/due date, så det er nemt at huske aflevering.

Afgrænsning

Projektet bliver udviklet som en MVP (Minimum Viable Product), hvilket betyder at vi fokuserer på de vigtigste funktioner, så løsningen kan demonstreres og testes i praksis.

- Vi afgrænser derfor følgende i første version:
- Vi laver ikke fuld forsikringshåndtering ved skader.
- Depositum registreres i systemet, men selve “skadeafgørelse” håndteres mellem brugere.
- Vi implementerer ikke nødvendigvis real online betaling (fx Stripe) i MVP.
- Hvis tiden ikke er til det, simulerer vi betaling/depositum som registrering i databasen.
- Vi håndterer ikke levering eller fragt – udgangspunktet er afhentning/aflevering fysisk mellem brugere.
- Vi laver ikke avanceret ID-verificering (fx NemID/MitID) i MVP.
- Vi fokuserer på de centrale flows: login, browse med lokation/filter, opret opslag, admin-godkendelse, booking/forespørgsel, chat og overblik over lejemål.

Afgrænsningen sikrer, at projektet bliver realistisk at færdiggøre inden for tiden og stadig opfylder kravene om flere teknologier, database, API og roller.

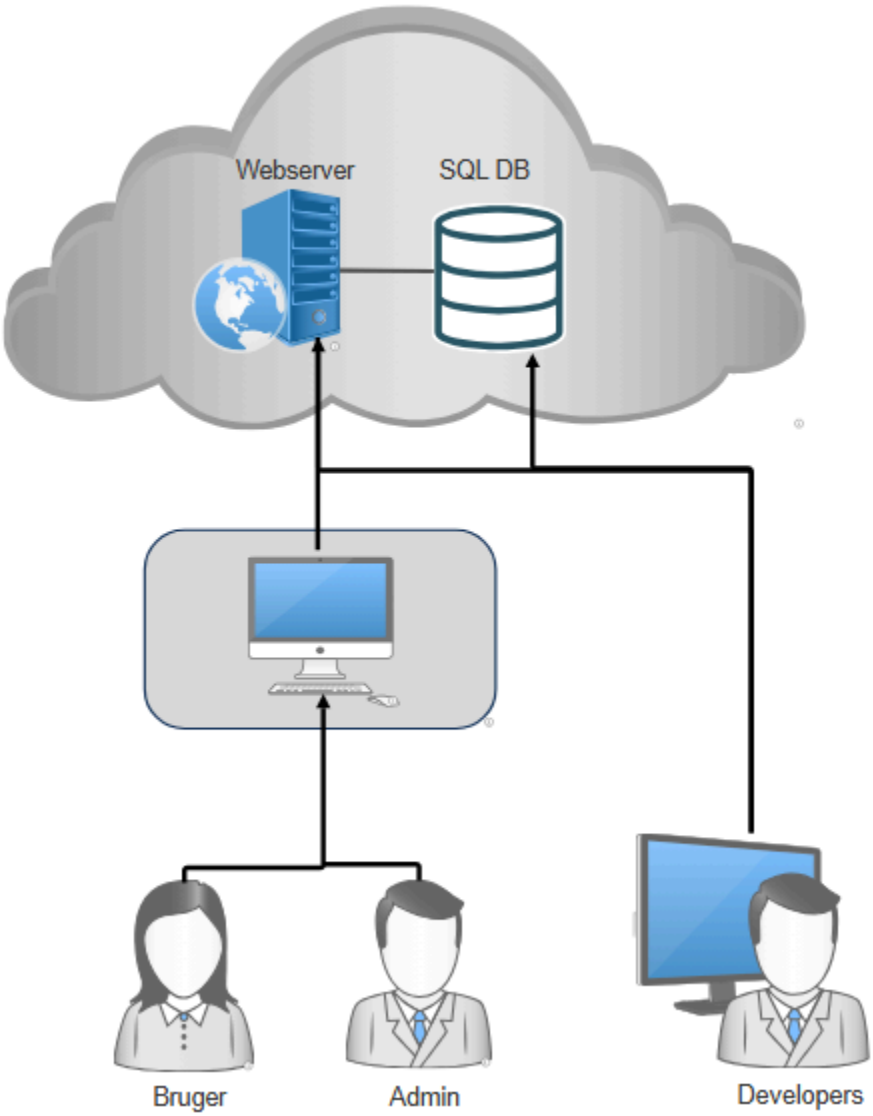
Fremtid (mulige udvidelser)

Når MVP’en fungerer, kan systemet udvides på flere måder. En naturlig udvidelse er at implementere en rigtig betalingsløsning, hvor både lejepris og depositum kan betales direkte i appen, og depositum automatisk kan frigives efter returnering.

Derudover kan man tilføje et vurderingssystem (rating/anmeldelser), så brugere kan opbygge troværdighed, og det bliver nemmere at vælge sikre udlejere. Man kan også udvide lokationsdelen med en rigtig kortvisning (Mapbox/Google Maps), bedre afstandsfiltrering og mulighed for at vise et “cirkelområde” omkring brugeren.

Andre relevante forbedringer er notifikationer (email/push) ved nye beskeder, godkendelser og kommende forfaldsdato, samt en rapporteringsfunktion, hvor brugere kan anmelde scam eller problematiske opslag. Til sidst kan admin-funktionaliteten udvides med automatiske regler (fx flagging af mistænkelige opslag) og bedre logning/overvågning, så platformen bliver mere robust ved mange brugere.

Aktørkontekst diagram



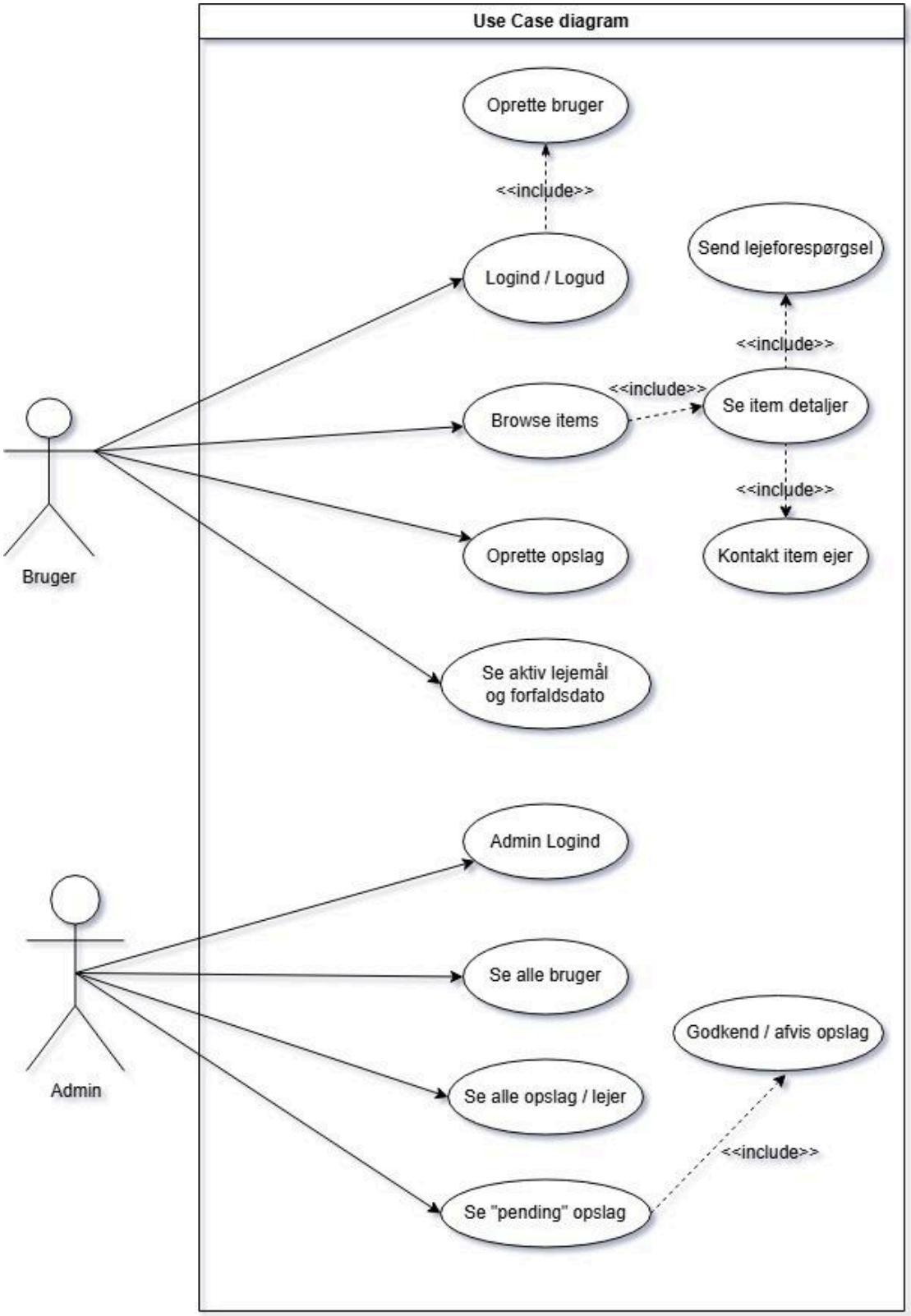
Aktør beskrivelse

Aktør navn	Bruger
Type	Primær aktør
Beskrivelse	Bruger er den almindelige bruger af systemet og kan både være lejer og udlejer. Brugeren kan oprette en konto, logge ind og oprette opslag for ting de vil udleje (fx plæneklipper, boremaskine osv.) med pris pr. dag og depositum. Brugeren kan også browse andre opslag, filtrere efter kategori, pris og især afstand/nær mig, så de kan finde ting tæt på deres lokation. Når brugeren finder et item, kan de sende en lejeforespørgsel og bruge chatten til at aftale detaljer som afhentning, tid og betaling/depositum. Brugeren skal kunne se egne opslag, forespørgsler og aktive lejemål med status og afleveringsdato.
Forventet teknisk niveau	Lavt til middel

Aktør navn	Administrator (Admin)
Type	Primær aktør
Beskrivelse	Administratoren er systemets moderator/superbruger og har ansvar for at sikre, at platformen fungerer sikkert og at indholdet er gyldigt. Admin kan logge ind i admin-panelet og gennemgå nye opslag, som brugere har oprettet, og derefter godkende eller afvise dem, før de bliver synlige i browse-sektionen. Admin kan også se alle brugere og inspicere deres opslag, lejeforespørgsler og lejeaftaler, så der kan følges op på misbrug, spam eller konflikter mellem brugere. Admin kan desuden håndtere rapporteringer/klager (hvis funktionen findes) og sikre, at systemets regler og retningslinjer bliver overholdt.
Forventet teknisk niveau	Middel

Aktør navn	Developers
Type	Sekundær aktør
Beskrivelse	Developers er dem der udvikler, vedligeholder og drifter løsningen. De har adgang til servermiljøet (webserver og database) for at kunne opsætte systemet, deploye nye versioner og overvåge stabilitet. Udviklere arbejder med fejlretning, forbedringer og opdateringer af både frontend og backend, samt database-migreringer og backup. De sørger også for sikkerhed i systemet, fx korrekt håndtering af login (hashing af passwords), API-sikkerhed, roller/rettigheder og at kommunikationen kører over HTTPS/SSL. Developers er ikke en almindelig brugerrolle i selve app'en, men en teknisk aktør der sikrer at systemet fungerer og kan videreudvikles.
Forventet teknisk niveau	Højt

Usecase Diagram



Krav om funktionalitet

For at systemet kan godkendes som en fungerende MVP, skal følgende funktioner være med. Kravene er opdelt i MK (Minimumskrav), K (Krav) og O (Optionelt), så det er tydeligt hvad der er “must-have” og hvad der kun kommer med, hvis der er tid.

Title	Type	Beskrivelse
Opret bruger og login	MK	Brugere skal kunne registrere sig, logge ind og logge ud. Roller skal understøttes (User/Admin).
Browse af opslag	MK	Brugere skal kunne se en liste over tilgængelige opslag (items) med grundinfo: titel, pris pr. dag og depositum.
Søgning og filtrering	MK	Brugere skal kunne søge på titel og filtrere på kategori. Der skal være en simpel “nær mig” funktion (afstand-filter), så brugeren kan se opslag tæt på deres lokation.
Opret opslag (udleje item)	MK	Brugere skal kunne oprette et opslag med titel, beskrivelse, kategori, pris pr. dag, depositum og lokation.
Admin-godkendelse af opslag	MK	Nye opslag skal få status Pending, og admin skal kunne godkende eller afvise. Kun Approved opslag vises i browse.
Lejeforespørgsel	MK	En bruger skal kunne sende en forespørgsel på et opslag og vælge en lejeperiode (fra/til dato). Systemet skal gemme status (fx Sent/Accepted/Declined).
Chat mellem bruger og udlejer	MK	Brugeren skal kunne chatte med udlejeren for at aftale afhentning og detaljer (minimum: tekstbeskeder).
Bruger-overblik (dashboard)	MK	Brugeren skal kunne se egne opslag (med status), egne forespørgsler og aktive lejemål med forfaldsdato/due date.
Admin-overblik	MK	Admin skal kunne se alle brugere og alle opslag/lejeforespørgsler i systemet.
Rediger/slet opslag	K	Brugeren skal kunne redigere eller slette egne opslag (med begrænsninger hvis det er aktivt udlejet).

Status på lejeaftale	K	Når en forespørgsel accepteres, skal der oprettes en aktiv leje, og item markeres som “ikke tilgængelig” i perioden.
Billed-upload	K	Brugeren skal kunne uploade mindst et billede til opslaget.
Notifikationer	K	Systemet viser beskeder som “opslag godkendt”, “ny chat besked”, “forespørgsel accepteret” (fx toast/badge).
Validering af input	K	Der skal være validering både i frontend og backend (fx pris > 0, depositum ≥ 0, datoer er gyldige).
Kortvisning	O	Map view med pins (Google Maps/Mapbox) ud fra opslagets lokation.
Rigtige betalinger	O	Integration med Stripe/MobilePay-lignende løsning til lejepris og depositum.
Ratings/anmeldelser	O	Efter endt leje kan brugere bedømme hinanden og item.
Rapportér opslag/bruger	O	Brugere kan anmelde mistænkelige opslag (admin kan håndtere rapporter).
Automatiske påmindelser	O	Påmindelse før forfaldsdato (fx e-mail eller push).

Case beskrivelser

Vores projekt handler om at lave en web app hvor folk kan leje og udleje ting til hinanden, som fx bøger, film, spil og andre små ting. Ideen er, at man ikke behøver at købe alt selv, men man kan låne/leje det af andre i en periode. Appen skal være nem at bruge, og det skal føles trygt, så man ikke bare mister sine ting.

Når en bruger laver en konto og logger ind, kan de både lægge deres egne ting op til udlejning og også leje ting fra andre. Hvis jeg fx har en bog, jeg ikke bruger lige nu, kan jeg oprette den som et “rent item” i systemet. Jeg skriver titel, beskrivelse, kategori (bog/film/værktøjer/andet), og hvor meget depositum der skal betales. Men den kommer ikke direkte ud til alle med det samme, fordi vi har en admin.

Admin skal nemlig først tjekke om det giver mening og om det er ok indhold. Det er fx for at undgå fake opslag, mærkelige billeder eller at folk skriver noget der ikke passer. Når admin har godkendt produktet, så bliver det synligt for andre brugere, og så kan de finde det, når de browser eller søger.

Når en anden bruger ser en ting de vil leje, så kan de sende en leje-forespørgsel. Her vælger de fx periode (fra dato til dato), og så betaler de et lille depositum (eller det bliver bare registreret i systemet, alt efter hvordan vi laver det). Efter det skal forespørgslen accepteres (det kan være af udlejer eller admin, afhængig af hvordan vi designer flowet). Når det er accepteret, så bliver det en aktiv leje, og systemet giver en due date, så man kan se hvornår man skal aflevere tilbage.

Teknologier

- Frontend: Angular
- Backend: ASP.NET Core Web API (C#)
- Database: SQL Server
- Login & roller: ASP.NET Identity + JWT (så vi kan styre User/Admin)
- Docker: Docker Compose så hele projektet kan køre samlet (frontend, api, database)
- Swagger: til at teste og vise API endpoints
- Github: Samarbejde og Versionsstyring

Systembeskrivelse

Webappen skal bygges som en Single Page Application (SPA), hvor frontend kommunikerer med en backend via et API. Systemet skal have en login-løsning med brugerhåndtering og mindst to roller: User og Admin. Brugere skal kunne oprette konto selv og oprette opslag/items til udlejning, men alle nye opslag skal igennem en admin-godkendelse, før de bliver vist offentligt i browse-sektionen. Alle data (brugere, items, leje forespørgsler og lejeaftaler) skal gemmes i en relationel database. Løsningen skal kunne køres lokalt, og hele projektet skal kunne startes samlet ved hjælp af Docker Compose, så frontend, backend og database kører sammen.

Funktionen

Systemet skal kunne:

- håndtere login og roller (User/Admin)
- lade brugere oprette items/opslag til udlejning
- vise kun godkendte opslag i browse
- lave lejeforespørgsler med depositum
- vise aktive lejemål og due dates
- give admin et panel til kontrol og godkendelse

Programmets begrænsninger

Systemet håndterer ikke levering/fragt, og vi antager at udlevering sker fysisk mellem brugere. Betaling af depositum kan enten være simuleret (registreret som transaktion i databasen) eller som en simpel integration.

Programmets fremtid

Senere kan systemet udvides med rating/anmeldelser, chat mellem lejer og udlejer, rigtige online betalinger (Stripe), notifikationer via e-mail/push og bedre filtrering (lokation, afstand osv.).

Brugerprofil

- User: almindelig bruger som udlejer og lejer items. Brugeren forventer at appen er nem og hurtig at bruge.
- Admin: person der modererer indhold, godkender opslag og kan inspicere alle data for at undgå misbrug.

Krav til udviklingsforløbet

Udviklingen foregår i en gruppe med versionsstyring via Git/GitHub. Arbejdet opdeles i opgaver/user stories, og der arbejdes iterativt (fx sprint/kanban). API dokumenteres i Swagger, og der laves minimum nogle tests af centrale flows (roller + godkendelse + leje).

Frontend Krav

- Login/Signup
  - Brugeren kan oprette konto og logge ind.
  - Brugeren forbliver logget ind via token/cookies.
- Browse sektion
  - Viser kun godkendte items.
  - Søgning + filtrering (kategori, titel, tilgængelighed).
- Item side (detaljer)
  - Viser info, billeder, depositum og status (tilgængelig/udlånt).
  - Knappen “Send lejeforespørgsel”.
- Opret item
  - Formular med titel, kategori, beskrivelse, billeder, depositum og tilgængelighed.
  - Når item oprettes, får det status Pending.
- Bruger dashboard
  - “Mine opslag” (Pending/Approved/Rejected)
  - “Mine lejeforespørgsler” (Sendt/Godkendt/Afvist)
  - “Mine aktive lejemål” med due date
- Admin panel
  - Godkendelses-kø for nye items (Approve/Reject + kommentar)
  - Oversigt over alle brugere
  - Admin kan se alle items og lejeaftaler på tværs af brugere

Backend krav

- Backend skal implementeres i ASP.NET Core Web API (.NET 8).
- Entity Framework Core skal bruges til databaseadgang.
- Auth & sikkerhed
  - Passwords skal hashes (Identity).
  - API endpoints skal beskyttes med JWT (eller en tilsvarende sikker løsning).
  - Rollebaseret adgang: admin endpoints må kun tilgås af Admin.
- Endpoints (minimum)

    - Auth: register/login/logout/refresh
    - Items: create/read/update/delete (med rolle checks)
    - Admin approval: get pending, approve, reject
    - Rental requests: create, view mine, approve/decline, status ændringer
    - Rentals: aktive lejemål + due date + afslut/return

Specifikke teknologi krav

- Angular til frontend.
- C# + ASP.NET Core Web API til backend.
- EF Core til ORM.
- SQL Server til database.
- Swagger til dokumentation og test af API.
- Docker Compose til at starte hele projektet samlet.

Ikke Funktionelle krav

- Skal virke på mobil, tablet og computer (responsive design).
- UI skal være overskueligt og ensartet (samme tema/komponenter).
- Koden skal være struktureret (services, DTO'er, tydelige lag), så den er nem at udvide og forklare til eksamen.
- Systemet skal håndtere fejl pænt (validering, meningsfulde fejlbeskeder).
- Databeskyttelse: brugerdata håndteres sikkert, og adgang styres via roller.

Testing

Unit testing (backend) Vi laver unit tests i xUnit på service-laget, så vi kan teste logikken uden UI. Det vigtigste at teste er de flows der kan give fejl eller sikkerhedsproblemer:

- Opret opslag → status bliver Pending
- Admin approve/reject → status bliver accepted og browse viser kun Approved
- Opret lejeforespørgsel → status "Sent"
- Accept leje → Rental oprettes med due date
- Rolle-adgang (User må ikke kalde admin endpoints)

Integration testing

Vi laver simple integration tests mod API'et (fx med TestServer/WebApplicationFactory), så vi tester:

- Auth flow (register/login + token virker)
- Protected endpoints returnerer 401/403 når man ikke er logget ind / ikke er admin

- CRUD på items virker end-to-end

User testing

Vi laver en kort brugertest med fx 3-5 personer (klassekammerater), hvor de får nogle opgaver:

- Opret bruger og log ind
- Opret et opslag
- Find et item i browse og send forespørgsel
- Find “mine lejemål” og se due date

Vi noterer hvor de bliver forvirrede (navigation, knapper, tekster), og vi retter UI bagefter.

Sikkerhed (login, hashing, kryptering, SSL)

Password hashing og login-sikkerhed

Vi bruger ASP.NET Identity, som automatisk hasher adgangskoder (så de aldrig gemmes i klar tekst). Det betyder, hvis nogen får adgang til databasen, kan de ikke bare læse passwords direkte. Vi sætter også password-krav (min længde, store/små bogstaver, tal osv.), så svage passwords undgås.

JWT + rollebaseret adgang

API’et beskyttes med JWT (eller secure cookies), og alle admin endpoints kræver admin-rolle. Hvis en user prøver at tilgå admin endpoints, skal systemet returnere 403 Forbidden. Det er også noget vi tester.

Kryptering af data (database)

Sensitive ting (fx tokens, evt. depositum-transaktioner) håndteres sikkert. Hvis der er behov, kan vi bruge kryptering af bestemte felter, men vigtigst er, at adgangskoder er hashed og at adgang til databasen er begrænset til serveren. (I en “rigtig” drift kan man også bruge database-kryptering som fx TDE i SQL Server, men i skoleprojekt beskriver vi det som en mulighed.)

SSL/HTTPS certifikat

Når systemet kører online, skal det køre via HTTPS med et SSL-certifikat, så data mellem bruger og server er krypteret (fx login og tokens). I udvikling bruger vi udviklingscertifikat, og i produktion kan man bruge et rigtigt certifikat (fx via Let's Encrypt). Det gør også at man undgår "sniffing" på netværket.

Ekstra sikkerhed

- Input-validering på backend
- Rate limiting / lockout ved mange login-forsøg (valgfrit men godt)
- Logging på serveren (så vi kan se fejl og forsøg på misbrug)

Kvalitetsfaktor

Der skal være fokus på sikker login, korrekt rolle-adgang, og at systemet ikke viser ikke-godkendte opslag offentligt. Koden skal være organiseret, så man kan forklare den (services, DTO'er, controller-lag osv)

Use case

Brugsscenarie: Opret bruger (sign up)
Aktør: User
Beskrivelse: Brugeren opretter en ny konto på platformen.
Forudsætning: Brugeren er ikke logget ind og har ikke allerede en konto med samme email.
Forventet: En ny bruger bliver oprettet og kan logge ind.
Forløb: 1. Brugeren går ind på "Sign up" siden. 2. Brugeren udfylder navn, email og adgangskode. 3. Systemet validerer felterne (fx email-format og password-krav). 4. Systemet opretter brugeren i databasen. 5. Brugeren får bekræftelse og kan logge ind (eller bliver logget ind automatisk).
Alternativt forløb: 3a. Brugeren udfylder ikke alle felter → fejlbesked vises. 3b. Adgangskoden opfylder ikke krav → fejlbesked vises, bruger bliver ikke oprettet. 4a. Email findes allerede → fejlbesked vises.

Brugsscenarie: Login
Aktør: User, Admin
Beskrivelse: Aktøren logger ind med email og adgangskode.
Forudsætning: Aktøren er oprettet i systemet.
Forventet: Aktøren bliver logget ind og får adgang baseret på rolle.
Forløb: 1. Aktøren går ind på “Login” siden. 2. Aktøren indtaster email og adgangskode. 3. Aktøren klikker “Login”. 4. Systemet validerer login-oplysninger. 5. Systemet udsteder token/session og logger aktøren ind. 6. Aktøren bliver sendt til startside/dashboard.
Alternativt forløb: 4a. Email eller password er forkert → fejlbesked og ingen login. 4b. Brugeren er låst/blokeret (hvis I laver det) → fejlbesked.

Brugsscenarie: Opret opslag (item til udlejning)
Aktør: User
Beskrivelse: Brugeren opretter et nyt opslag (bog/film/andet) til udlejning.
Forudsætning: Brugeren er logget ind.
Forventet: Opslaget oprettes med status Pending, og admin kan se det til godkendelse.
Forløb: 1. Brugeren går til “Opret opslag”. 2. Brugeren udfylder titel, kategori, beskrivelse, depositum og evt. billede. 3. Brugeren klikker “Opret”. 4. Systemet validerer input. 5. Systemet gemmer opslaget med status Pending. 6. Brugeren kan se opslaget under “Mine opslag” som Pending.
Alternativt forløb: 4a. Manglende/ugyldige felter → fejlbesked. 5a. Fejl ved upload/gem → systemet viser fejl og opslag oprettes ikke.

Brugsscenarie: Admin godkender/afviser opslag
Aktør: Admin
Beskrivelse: Admin gennemgår nye opslag og godkender eller afviser dem.
Forudsætning: Admin er logget ind og der findes mindst ét Pending opslag.
Forventet: Opslagets status bliver ændret til Approved eller Rejected.
Forløb: 1. Admin går ind på “Admin panel”. 2. Admin åbner listen “Pending opslag”. 3. Admin vælger et opslag og gennemgår informationen. 4. Admin klikker “Approve” eller “Reject” (evt. med kommentar). 5. Systemet opdaterer status på opslaget. 6. Hvis Approved → opslaget bliver synligt i browse-sektionen.
Alternativt forløb: 2a. Der findes ingen Pending opslag → systemet viser “Ingen nye opslag”. 5a. Database/serversvar fejler → fejlbesked og status ændres ikke.

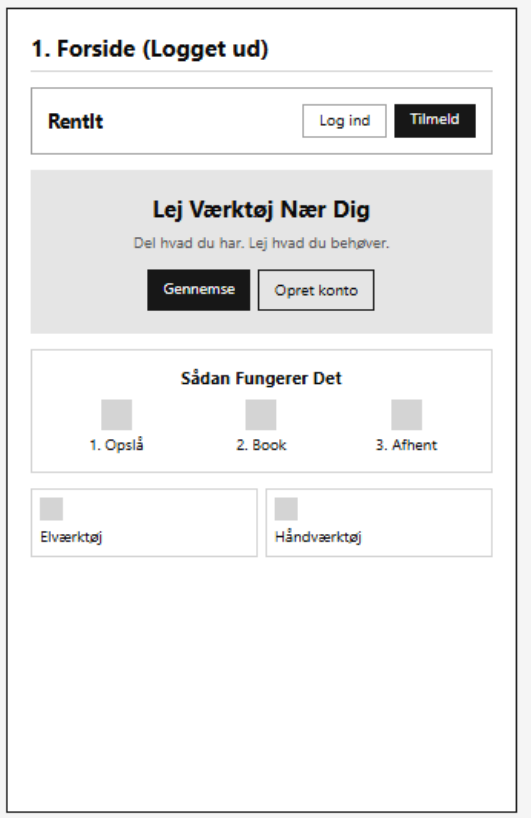
Brugsscenarie: Browse og søge efter items
Aktør: User
Beskrivelse: Brugeren browser og søger efter godkendte items.
Forudsætning: Der findes Approved items i systemet.
Forventet: Brugeren kan se en liste og filtrere/søge.
Forløb: 1. Brugeren går ind på “Browse”. 2. Systemet viser kun items med status Approved. 3. Brugeren søger på titel eller filtrerer på kategori. 4. Systemet opdaterer listen med matchende items. 5. Brugeren klikker på et item for at se detaljer.
Alternativt forløb: 2a. Ingen Approved items → “Der er ingen opslag endnu”. 4a. Ingen resultater → “Ingen match”.

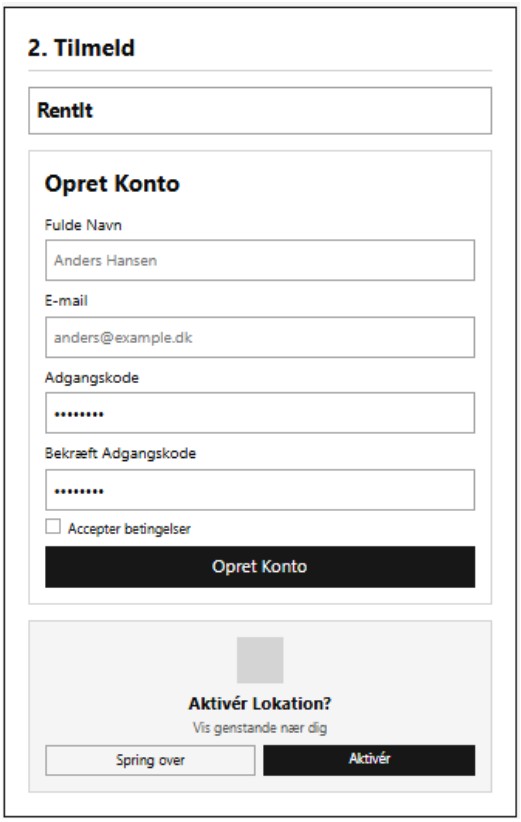
Brugsscenarie: Send lejeforespørgsel
Aktør: User (lejer)
Beskrivelse: Brugeren sender en forespørgsel om at leje et item og betaler/registrerer depositum.
Forudsætning: Brugeren er logget ind og item er Approved og tilgængelig.
Forventet: Der oprettes en lejeforespørgsel med status Sent.
Forløb: 1. Brugeren åbner et item. 2. Brugeren klikker “Send lejeforespørgsel”. 3. Brugeren vælger periode (start/slut) og bekræfter depositum. 4. Systemet opretter en RentalRequest med status Sent. 5. Brugeren kan se den under “Mine lejeforespørgsler”.
Alternativt forløb: 3a. Ugyldig periode (slutdato før startdato) → fejlbesked. 4a. Item er blevet udlånt i mellemtiden → systemet afviser og viser besked.

Brugsscenarie: Godkend lejeforespørgsel (opret aktiv leje)
Aktør: Admin
Beskrivelse: En lejeforespørgsel godkendes, og der oprettes en aktiv leje med due date.
Forudsætning: Der findes en RentalRequest med status Sent.
Forventet: Status bliver Accepted, og en Rental oprettes med due date
Forløb: 1. Admin åbner lejeforespørgsler i admin panel (eller udlejer ser forespørgslen). 2. Admin vælger forespørgslen. 3. Admin klikker “Accept”. 4. Systemet opretter en Rental med startdato og due date. 5. Systemet ændrer item til “udlånt/ikke tilgængelig”. 6. Lejer kan nu se aktiv leje under “Mine lejemål”.
Alternativt forløb: 3a. Admin klikker “Decline” → status bliver Afvist, ingen Rental oprettes. 4a. Systemfejl → status ændres ikke.

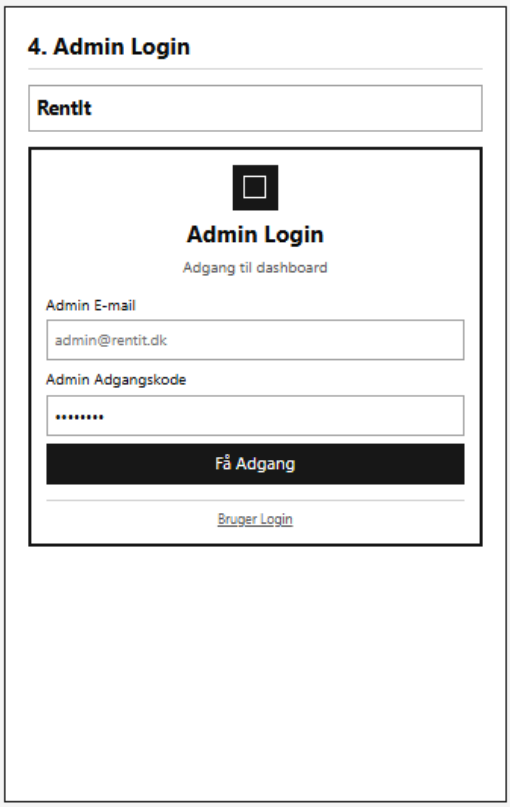
Brugsscenarie: Se mine aktive lejemål og due date
Aktør: User
Beskrivelse: Brugeren kan se hvilke ting de lejer, og hvornår de skal afleveres.
Forudsætning: Brugeren er logget ind og har mindst én aktiv leje.
Forventet: Brugeren ser en liste med items med due date.
Forløb: 1. Brugeren går ind på “Dashboard”. 2. Brugeren åbner “Mine aktive lejemål”. 3. Systemet viser liste med item, startdato og due date. 4. Brugeren kan klikke ind på detaljer.
Alternativt forløb: 2a.Ingen aktive lejemål → “Du har ingen aktive lejemål”.

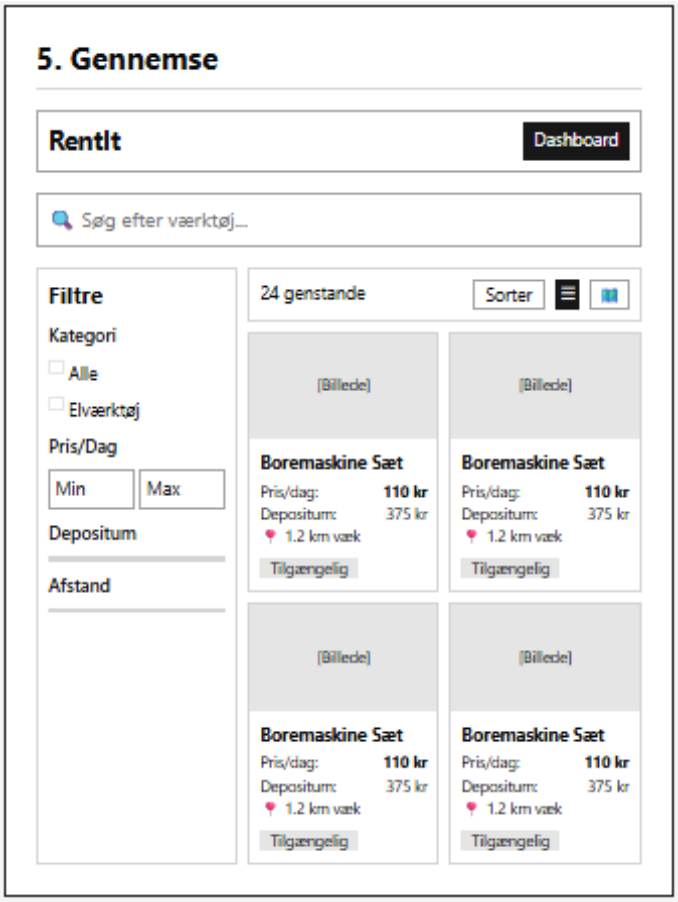
Wireframes

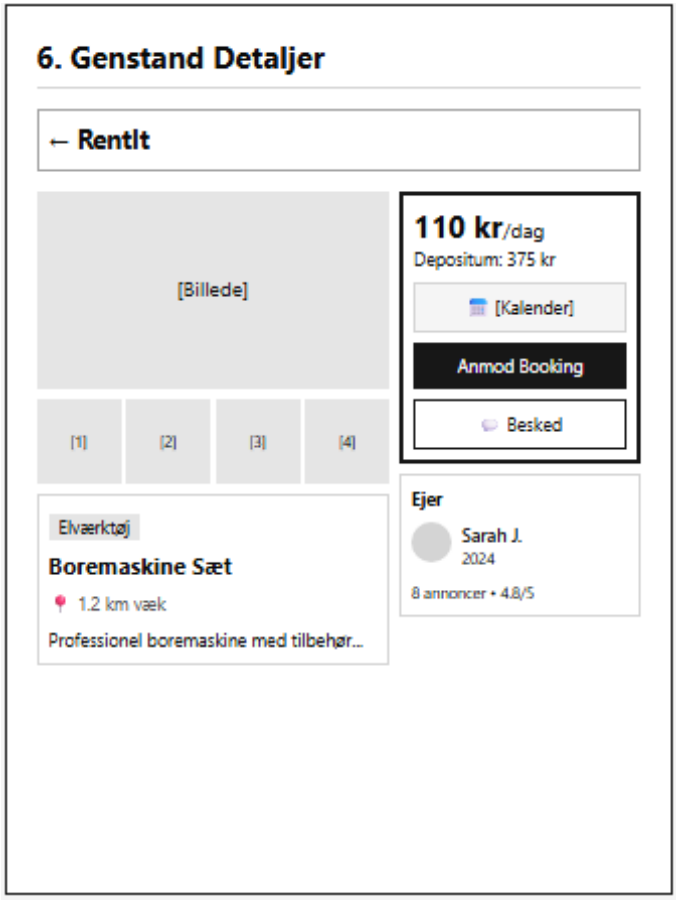

1. Forside (Logget ud)
RentIt
Log ind
Tilmeld
Lej Værktøj Nær Dig
Del hvad du har. Lej hvad du behøver.
Gennemse
Opret konto
Sådan Fungerer Det
1. Opslå
2. Book
3. Afhent
Elværktøj
Håndværktøj


2. Tilmeld
RentIt
Opret Konto
Fulde Navn
Anders Hansen
E-mail
anders@example.dk
Adgangskode
Bekræft Adgangskode
Accepter betingelser
Opret Konto
Aktivér Lokation?
Vis genstande nær dig
Spring over
Aktivér

3. Login (Bruger)
RentIt
Velkommen Tilbage
Log ind på din konto
E-mail
anders@example.dk
Adgangskode
Glemt adgangskode?
Log Ind
Har du ikke en konto? Tilmeld
Admin Login


4. Admin Login
RentIt
Admin Login
Adgang til dashboard
Admin E-mail
admin@rentit.dk
Admin Adgangskode
Få Adgang
Bruger Login


5. Gennemse
RentIt
Dashboard
Søg efter værktøj...
Filtre
Kategori
Alle
Elværktøj
Pris/Dag
Min
Max
Depositum
Afstand
24 genstande
Sorter
[Billede]
Boremaskine Sæt
Pris/dag: 110 kr
Depositum: 375 kr
1.2 km væk
Tilgængelig


6. Genstand Detaljer
← RentIt
[Billede]
[1]
[2]
[3]
[4]
110 kr/dag
Depositum: 375 kr
[Kalender]
Anmod Booking
Besked
Elværktøj
Boremaskine Sæt
1.2 km væk
Professionel boremaskine med tilbehør...
Ejer
Sarah J.
2024
8 annoncer • 4.8/5

7. Anmod Booking

Anmod Booking

Start Dato

15/03/2026

Slut Dato

17/03/2026

Besked til Ejer

Fortæl ejeren om dit projekt...

Booking Oversigt

110 kr/dag × 3 dage	330 kr
Depositum	375 kr
I Alt	**705 kr**

Annuller | Send Anmodning

8. Beskeder

Rentlt Dashboard

Beskeder

Sarah J.
Boremaskine
Afhentning kl. 10..

Sarah J.
Boremaskine
Afhentning kl. 10..

Sarah J.
Boremaskine
Afhentning kl. 10..

Sarah J.

Boremaskine Sæt
15-17 mar • 110 kr/dag • 375 kr depositum

Hej! Interesseret i weekenden

Ja, den er ledig!

Del lokation | Bekræft tid

Skriv besked...

9. Opslå Genstand (Multi-step)

← RentIt

Opslå Din Genstand

1 — 2 — 3 — 4

Info | Priser | Lokation | Fotos

Genstand Information

Titel *

f.eks. Boremaskine Sæt

Kategori *

Vælg kategori ▼

Stand *

Vælg stand ▼

Beskrivelse *

Beskriv din genstand...

Fortsæt

✓ **Annonce Indsendt!**

Afventer godkendelse

Gå til Dashboard

Opslå Endnu En

10. Mit Dashboard

RentIt + Opslå

Mit Dashboard

Mine Annoncer Mine Lejer Profil

Alle | Afventer | Godkendt | Afvist

Boremaskine Sæt Godkendt
110 kr/dag

Haveslange 15m Afventer
110 kr/dag

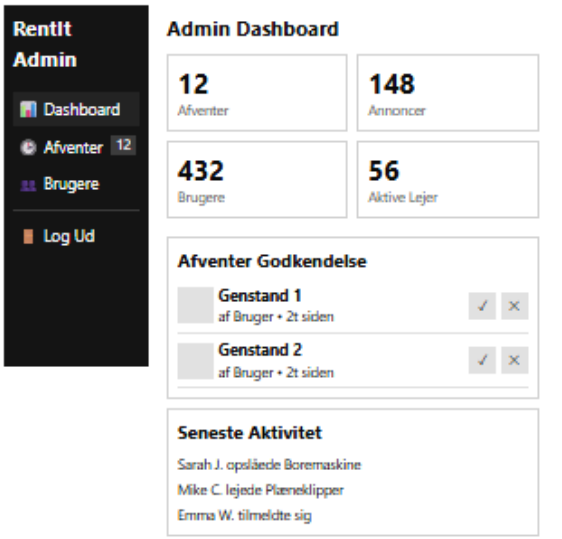
Kædesav Udlejet
110 kr/dag

Profil & Indstillinger

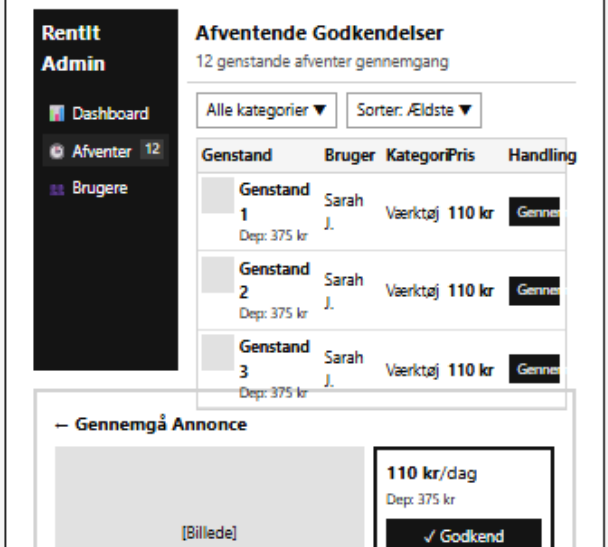
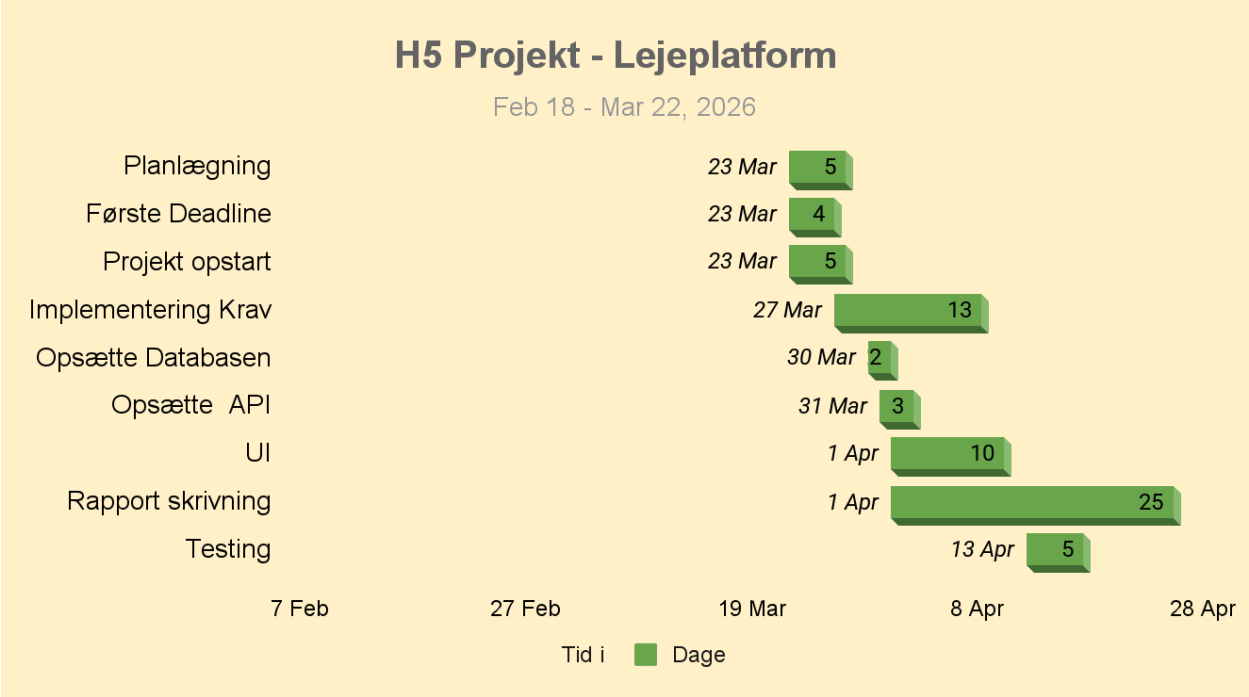
København, DK

Log Ud

11. Admin Dashboard



12. Afventende Godkendelser



GANTT Diagram